LA ROCHELLE UNIVERSITÉ

LICENSE 3 INFORMATIQUE


# Modélisation de bases de données

# Project Report:

# Designing Postgres Database for Genealogy dataset

**Professor(s)** BOUWMANS Thierry

**Members** TRAN Minh Duong

PHAM Ngan Ha

NGUYEN Hoang Nhat

# Abstract

This project builds a database to organize and study marriage records from France. We take 5,000 marriage documents and create a PostgreSQL database to store them in an organized way. The goal is to design the database so that it is well-organized, easy to use, and can answer questions about families and dates. This report explains how we organized the data and built the database system.


# I. Introduction

Old documents and records from France contain important information about families and history. Many of these documents are stored in archives. To study and understand this information, we need to organize it in a database. This project focuses on creating a database system for marriage records from the Vendée region. We show how to take messy information and organize it in a clean, useful way.

## I.1. Dataset Overview

We use a file called `mariages_L3_5k.csv` which contains 5,000 marriage records. Each record includes information about two people who got married, their parents, and the place and date of marriage.

The data has these main parts:

- **Record Information:** Each record has an ID number and a date
- **Person A Information:** Name of the first person, and names of their mother and father
- **Person B Information:** Name of the second person, and names of their mother and father
- **Location Information:** The city (commune) and region (department) where the marriage happened

## I.2. Why This Project Is Important

When we have many old documents, it is difficult to find information or see patterns. A database makes this easier. Our database has these goals:

1. **Organization:** Store all the information in one place in a clear way
2. **No Duplicates:** Each person should appear only once in the database, even if they appear in many records
3. **Correct Information:** Make sure that all relationships between people and places are correct

# II. Database Design & Normalization

## II.1. Initial Conceptualization

Initially, we treat the dataset as a single flat table, where the Act ID is the primary key and every other attribute depends on it

Corresponding FD:

```
Act ID → (
    Act Type,
    Person A's Last Name,
    Person A's First Name,
    First Name of A's Father,
    Last Name of A's Mother,
    First Name of A's Mother,
    Person B's Last Name,
    Person B's First Name,
    First Name of B's Father,
    Last Name of B's Mother,
    First Name of B's Mother,
    Town,
    Department,
    Date,
    View Number
)
```

=> Result: 1NF achieved

## II.2. Normalization Process

1. **For 1NF:**
   All values are atomic (each cell contains one piece of data).

   Each record is uniquely identified by an Act ID => Result: 1NF achieved

2. **For 2NF:**
   **Problem**: Repetition of data for parents, towns, and departments leads to redundancy.

   **Solution**: Separate entities so that every non-key attribute depends on the whole primary key

   **Resulting FDs**:

   ```
   - Act_ID → (Type, Date, View_Num, Commune_ID, Person_A_ID, Person_B_ID)
   - Person_ID → (Last_Name, First_Name, Father_ID, Mother_ID)
   - Commune_ID → (Name, Dept_Code)
   ```

   => Result: 2NF achieved

3. **For 3NF:**
   **Problem**:

   - Transitive Dependency: The Department depended on the Town, which depended on the Act.
   - Parental names depended on the person

   **Solution**:

   - Extracted Department into a standalone table to enforce valid codes (44, 49, 79, 85).
   - Established foreign keys `(father_id, mother_id)` within the Person table to handle lineage without repeating names.

   **Final FDs**:

```
Act_ID → (Act_Type, Date, View_Num, Commune_ID, Person_A_ID, Person_B_ID)
Person_ID → (Last_Name, First_Name, Father_ID, Mother_ID)
Commune_ID → (Commune_Name, Dept_Code)
Dept_Code → (Valid_Codes: 44, 49, 79, 85)
```

   => Result: 3NF achieved

## II.3. Final Relation Schema

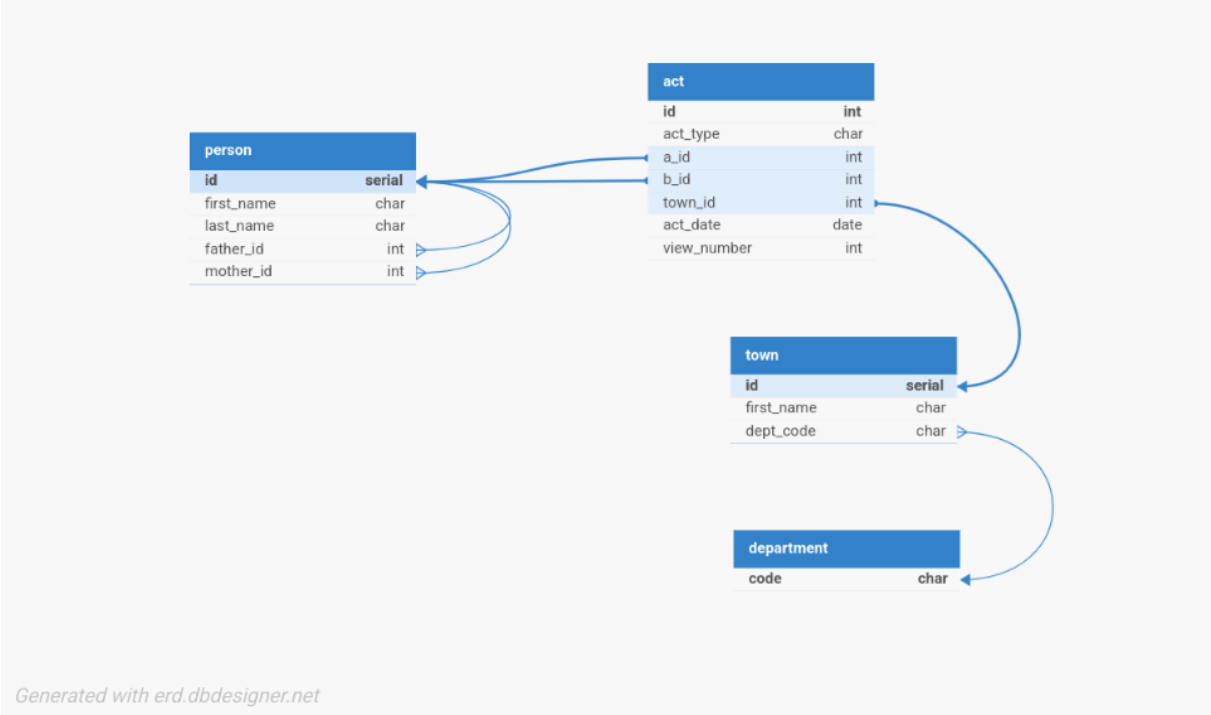


Figure 1: Database Relation Schema

**Cardinality:**

- Each parent can have more than one child but a child can only have one of each parent.

- Each department can have more than one town, but a town must only belong to one department.

## III. Data Extraction & ETL Process

TODO:
Cleaning strategy: French character encoding?
Command Line Tools with cut sort uniq?
Import to Postgres with \copy

## IV. Genealogical Queries & Results

1. The number of communes per department
2. The number of records in Luçon
3. The number of marriage contracts before 1855
4. The commune with the highest number of marriage banns
5. The date of the first and last records

Query? DELET FROM ProdDB; xD

## V. Bonus: Scalability & Noisy Data

TODO:
Challenge Overview: Dealing with the 564k register mariages.csv file .
Difficulties Encountered: Description of the "noise" found and how you resolved it.
Performance Comparison: Comparison of query results between the 5k and 564k datasets.

## VI. Conclusion